

# Design Exercise: Space Software Engineer

Transcelestial Technologies

---

## What we're looking for

This exercise asks you to do two things a strong engineer on our team does regularly: architect a real-time system, and write the hard code at the centre of it. Just as important to us is how you think through the problem, so please show your reasoning, the trade-offs you weigh, and the options you decide against.

Treat this as a design you would defend in a technical review rather than a coding puzzle. Budget a few days for it. We are not testing speed. We care about depth, judgement, and whether you take the work all the way to something that runs.

## The system: a PAT (Pointing, Acquisition and Tracking) terminal

Design the on-board software for a laser communications PAT terminal. The terminal has to acquire a counterpart terminal, lock onto it, and hold a tight pointing lock while both platforms move and vibrate.

**The system must be designed and implemented in ROS 2.** Our space stack is ROS 2 based, so please use ROS 2 nodes, topics and services, QoS, and executors as your building blocks throughout. The terminal has the following hardware for your software to orchestrate:

- **Camera / position sensing detector.** Sees the incoming beacon as a spot and gives a position error (where the beam landed versus where it should have). Moderate rate, in the tens of Hz, with higher latency.
- **IMU.** High rate angular rate and attitude data for the platform, from hundreds of Hz up to around 1 kHz, at low latency. Used to reject disturbance between camera updates.
- **Coarse actuator, a gimbal.** Wide range, slow, low bandwidth. Used to slew and acquire.
- **Fine actuator, a fast steering mirror (FSM).** Small range, very fast (around 1 kHz class), high bandwidth. Used to hold a tight lock and reject vibration.
- A link to the host or mothership that commands modes and receives telemetry.

## Part A: System architecture

Produce a design document that covers the following.

- **A ROS 2 node architecture, with a diagram.** Show all the nodes, what each one owns, and the topics and services between them. Cover sensing (camera, IMU), estimation and fusion, the two stage control (coarse gimbal and fine FSM), mode and state management, and telemetry and health.
- **The control hierarchy.** How coarse (gimbal) and fine (FSM) control coexist and hand off: acquisition, then hand over to fine lock, then holding lock. Explain what happens on loss of lock, and when the FSM saturates and runs out of range.
- **A timing and latency budget.** The rate each loop runs at, which paths are hard real time, and how you keep a slow or high latency path (such as the camera or vision) from contaminating a fast one (such as the FSM loop).
- **Your ROS 2 middleware choices.** For each significant topic or service, the QoS profile you would choose (reliability, durability, history, depth) and why. How you would structure ROS 2 executors and

callback groups so the kHz path is never starved. How nodes behave when they start in the wrong order (DDS discovery and late join).

- **Failure handling.** Sensor dropout (a lost camera frame, an IMU glitch), mode transitions, and safe states.

## Part B: Identify and defend the hardest part

List the three or four hardest or highest risk parts of this system, rank them, and make the case for the single one you consider hardest. We want your reasoning rather than a hedge, so defend why your top pick matters more than the others.

## Part C: Implement that piece

**Write runnable ROS 2 code for the piece you ranked hardest in Part B**, implemented as one or more ROS 2 nodes. It can run against simulated or synthetic inputs, so no real hardware is needed, but it should genuinely work and be something we can clone and run with ROS 2. Whatever you build, it should show that the hard part actually behaves correctly under the conditions that make it hard. For example, the handoff should actually hand off, the estimator should actually fuse, or the loop should actually hold timing. Use C++ or Python (rclcpp or rclpy), whichever lets you do your best work. Include the ROS 2 distro you targeted, run and launch instructions, and any tests or a way for us to check the behaviour.

## Part D: Your thinking

Throughout, show your work. Tell us the alternatives you considered and set aside, the assumptions you made, and what you would do next given more time or real hardware. We read this as closely as the code.

## Deliverables

- A design document covering Parts A, B and D, in any format. A diagram is expected for the architecture.
- A git repo or a zip with the Part C ROS 2 code, run and launch instructions, and any tests.
- A note of which ROS 2 distro you targeted.

*If a requirement is ambiguous, state your assumption and carry on. Making sensible assumptions explicit is part of the job. Ask us if you are genuinely blocked.*